Tutorial/Laboratory 04

Web Validation and Accessibility Testing

1. LEARNING OUTCOMES

Upon completion of this lesson, you should be able to:

- Validate HTML content using the W3C Markup Validation Service.
- Evaluate the accessibility of a website using the axe accessibility testing toolkit.

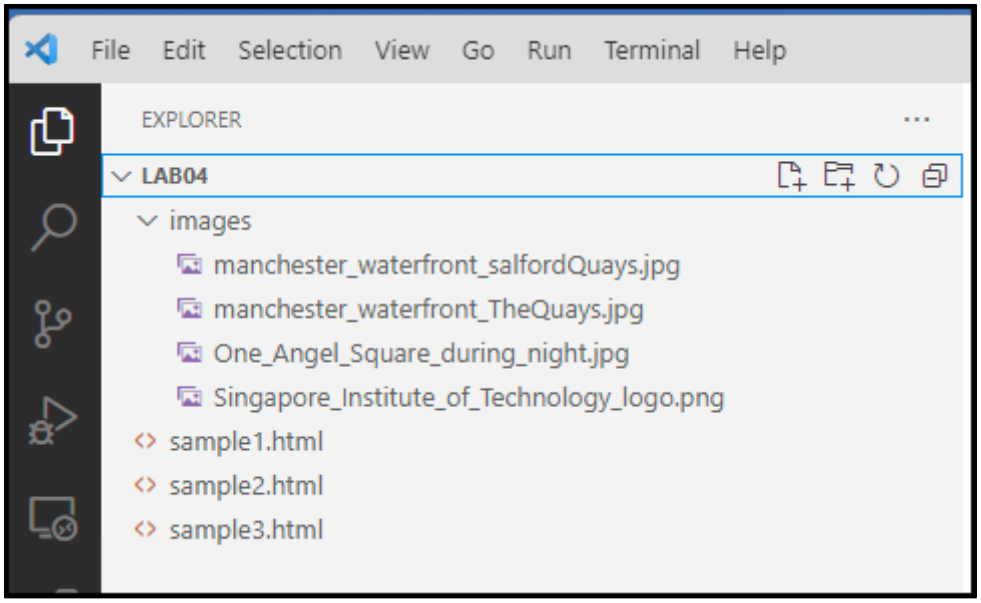
2. REQUIRED SOFTWARE

- Visual Studio Code (https://code.visualstudio.com/)
- Supported web browser (Chrome or FireFox recommended).
- W3C Markup Validation Service (https://validator.w3.org/)
- Deque's axe accessibility testing plugin
  - Firefox: https://addons.mozilla.org/en-US/firefox/addon/axe-devtools/
  - Chrome: https://chrome.google.com/webstore/detail/axe-web-accessibility-tes/lhdoppojpmngadmnindnejefpokejbdd

3. INITIAL SETUP

3.1 Before starting the exercises, perform the following steps:

a. Download the file "Lab04-Sample-Files.zip" from xSiTe, unzip and copy the contents into a new **Lab04** folder. Your folder contents should look like this:



b. Install the **axe** plugin using one of the links provided in section 2.

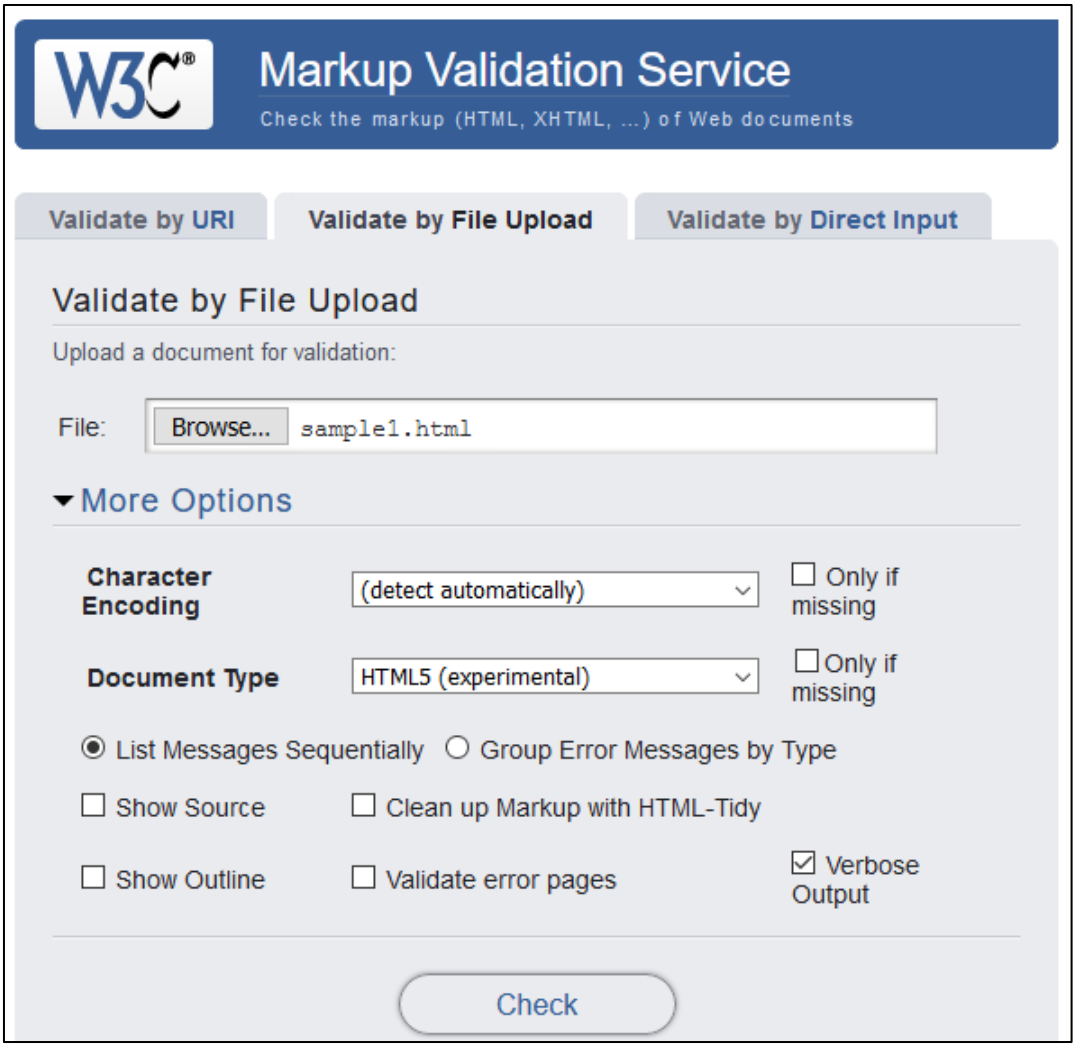
4. EXERCISE 1: WEB VALIDATION

4.1 Let's get some hands-on experience using the W3C Markup Validation Service. Navigate to https://validator.w3.org/ and observe the three tabs:

- **Validate by URI** - allows you to validate any publicly available web page online. For example, try typing www.singaporetech.edu.sg in the 'Address' box and hit the 'Check' button. You may see some interesting results!
- **Validate by File Upload** - allows you to upload a HTML document from your local file system and perform validation. This is the method we'll use in the exercise.
- **Validate by Direct Input** - allows you to type or paste HTML content directly in the editor and perform validation.

4.2 Go to the 'Validate by File Upload' tab. Click the 'Browse…' button, navigate to your **Lab04** folder, and select 'sample1.html' as the input file.

Expand the 'More Options' panel and select "HTML5 (experimental)" as the document type. Lastly, check the 'Verbose Output' checkbox to get more explanatory output:



4.3 Click the 'Check' button and observe the results. The validator should indicate 1 warning and 2 errors:

1. Warning **Consider adding a `lang` attribute to the `html` start tag to declare the language of this document.**

   From line 1, column 16; to line 2, column 6

   `TYPE html>↵<html>↵  <`

   For further guidance, consult Declaring the overall language of a page and Choosing language tags.

   If the HTML checker has misidentified the language of this document, please file an issue report or send e-mail to report the problem.

2. Error **End tag `header` seen, but there were open elements.**

   From line 12, column 13; to line 12, column 21

   `</header>↵↵`

3. Error **Unclosed element `h1`.**

   From line 11, column 17; to line 11, column 20

   `<h1>Explor`

4.4 Edit 'sample1.html' in VS Code and fix all of the problems. Run the file through the validator again to check that there are no errors:

Document checking completed. No errors or warnings to show.

Used the HTML parser.

Total execution time 2 milliseconds.

4.5 Now repeat the above steps with 'sample2.html' and then 'sample3.html' as the input files. Fix all errors in both files and verify that the updated files pass validation.

5. <u>HOW TO VALIDATE SERVER-SIDE FILES</u>

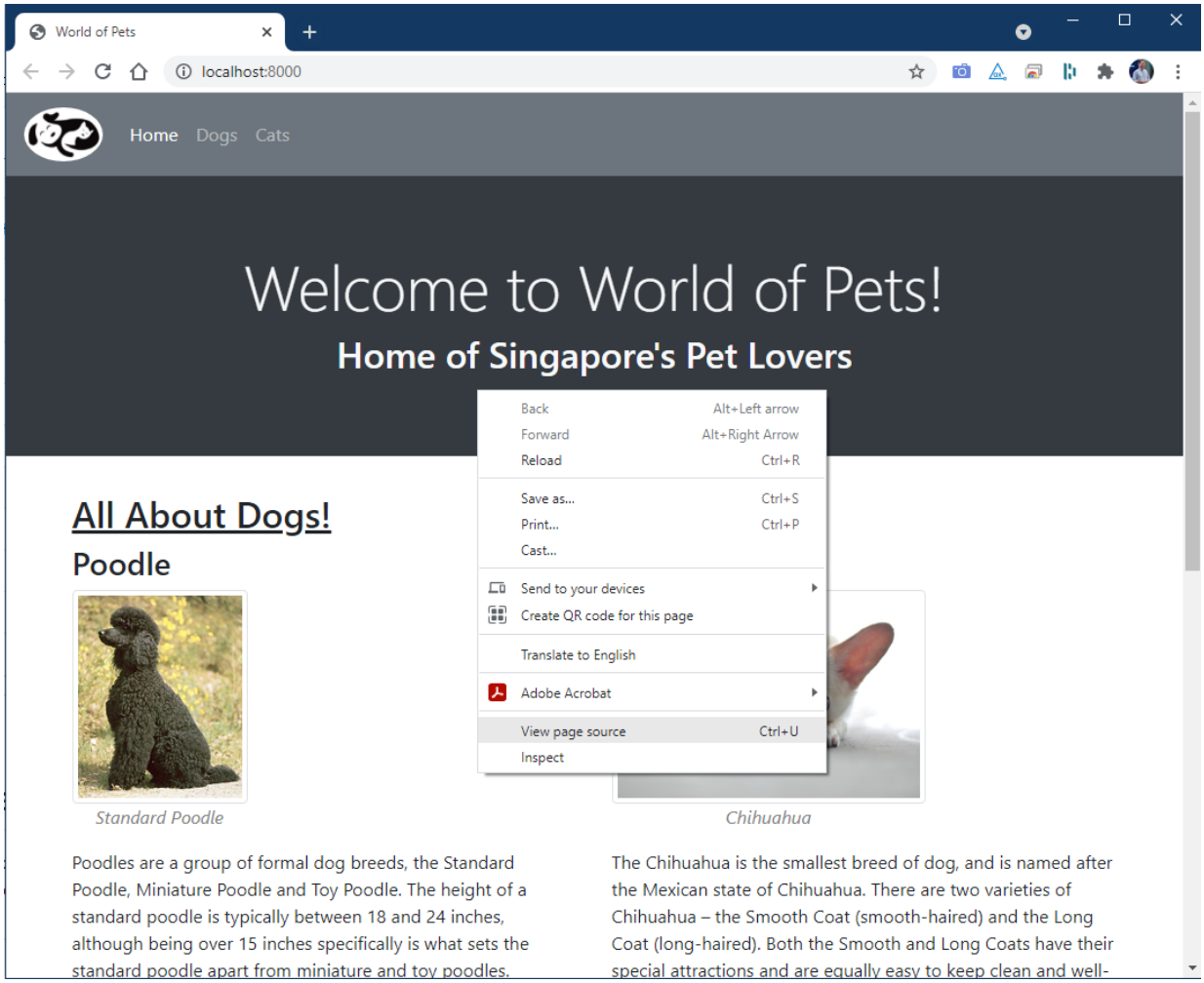
5.1 Note: this section is not part of the exercise, it is only for your reference when we study server-side processing in a future lesson.

5.2 Server-side files, such as PHP (.php) or ASP.NET (.aspx) source files **cannot** be uploaded to the W3C Markup Validation Service directly, as we did with the HTML files above. Instead, you must use either the 'Validate by URI' method if your website is publicly accessible, or the

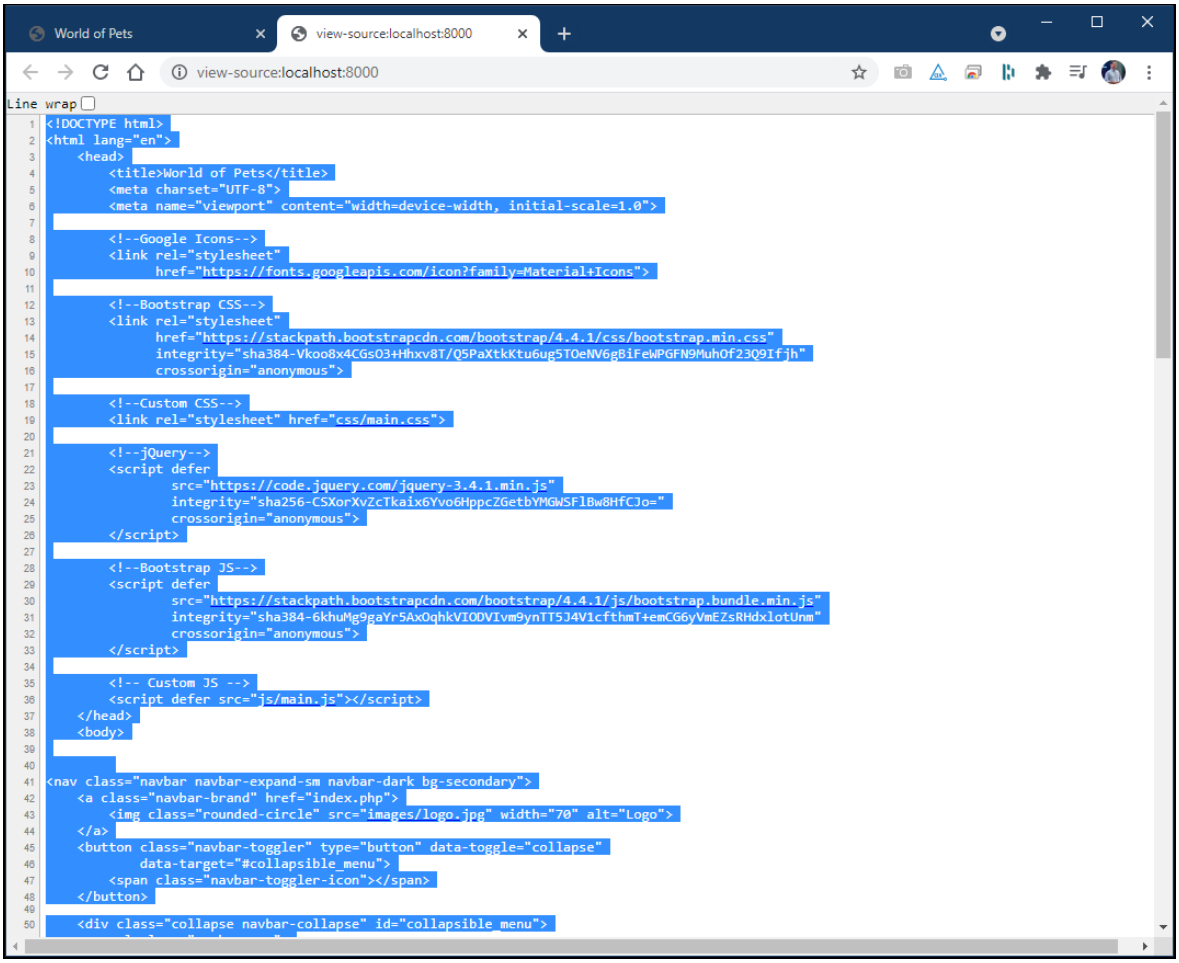
'Validate by Direct Input' method if your website is running on localhost. To use the latter method, you would:

a. Run the PHP page on localhost to generate a pure HTML page in the browser.

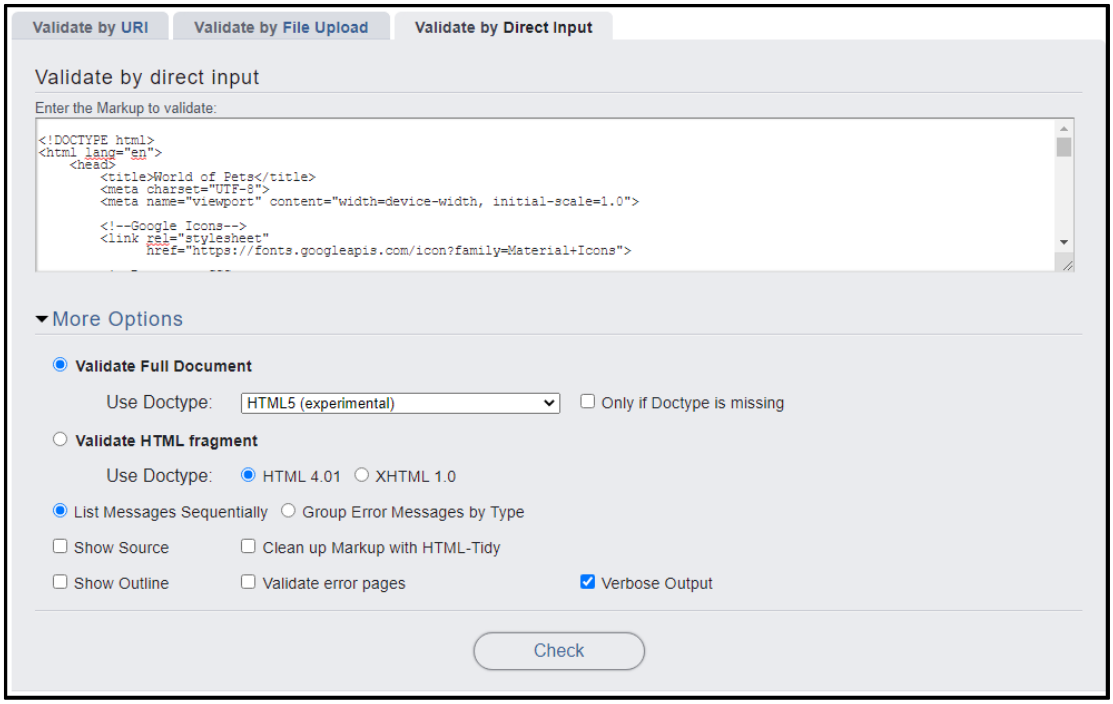
b. Right-click on the page and select 'View Page Source' from the popup menu:



c. Select all of the HTML code in the source view (CTRL-A) and copy to the clipboard (CTRL-C):



d. Go to the 'Validate by Direct Input' tab in the W3C Markup Validation Service and paste (CTRL-V) the content into the editor:



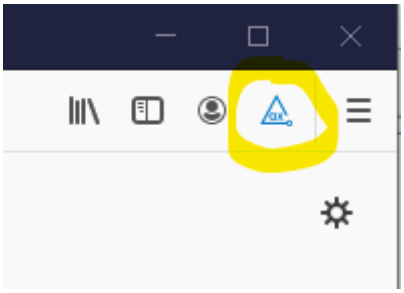
e. Click 'Check' to perform the validation and view the results:

6. EXERCISE 2: ACCESSIBILITY TESTING

6.1 The validation tests performed in exercise 1 will help you to correct HTML5 content problems, however they do not help very much with accessibility issues. For this, we need to conduct additional testing using a dedicated accessibility evaluation tool.

There are many such tools available (refer to https://www.w3.org/WAI/ER/tools/) and you are encouraged to experiment, however for this exercise we are going to use **axe**, one of the most popular accessibility testing tools available.

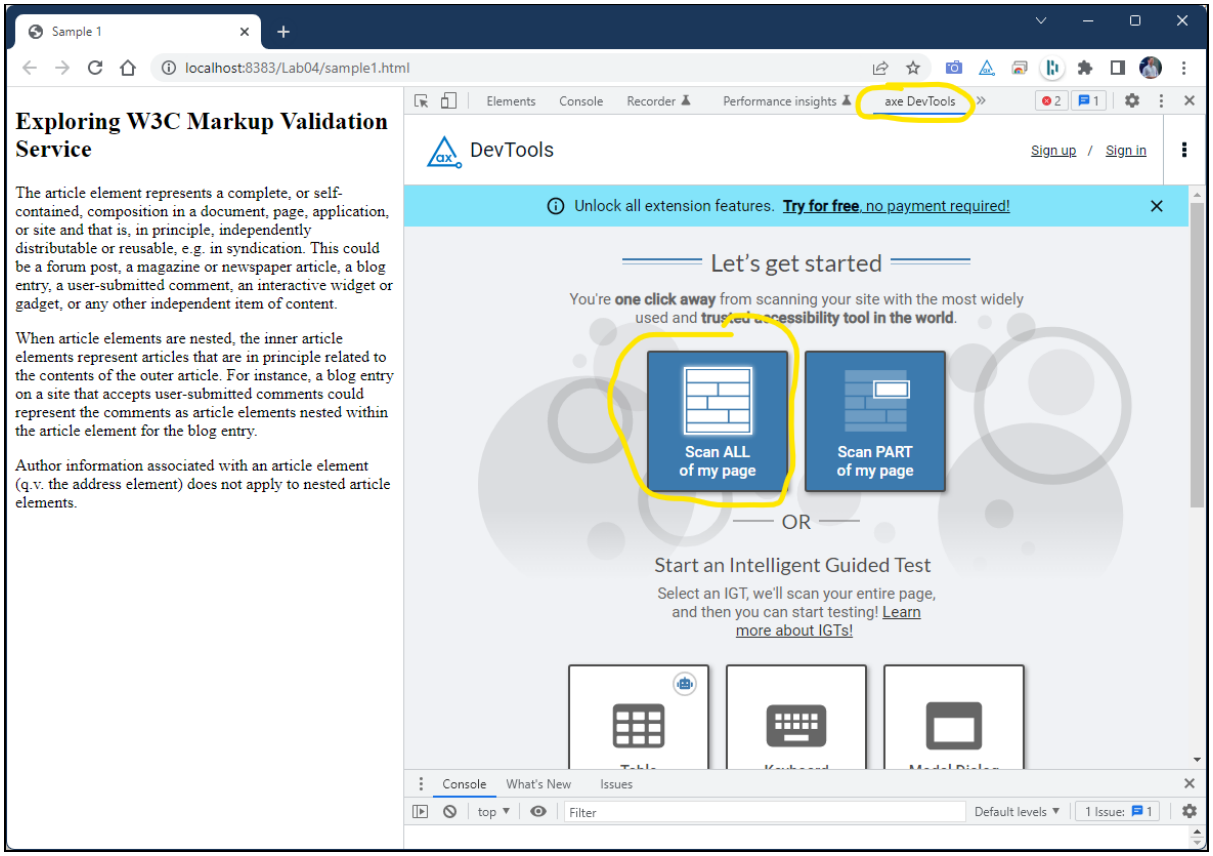
If you successfully installed either the Firefox or Chrome extension for axe as directed in section 3, you should see the axe icon in the upper right corner of your browser:



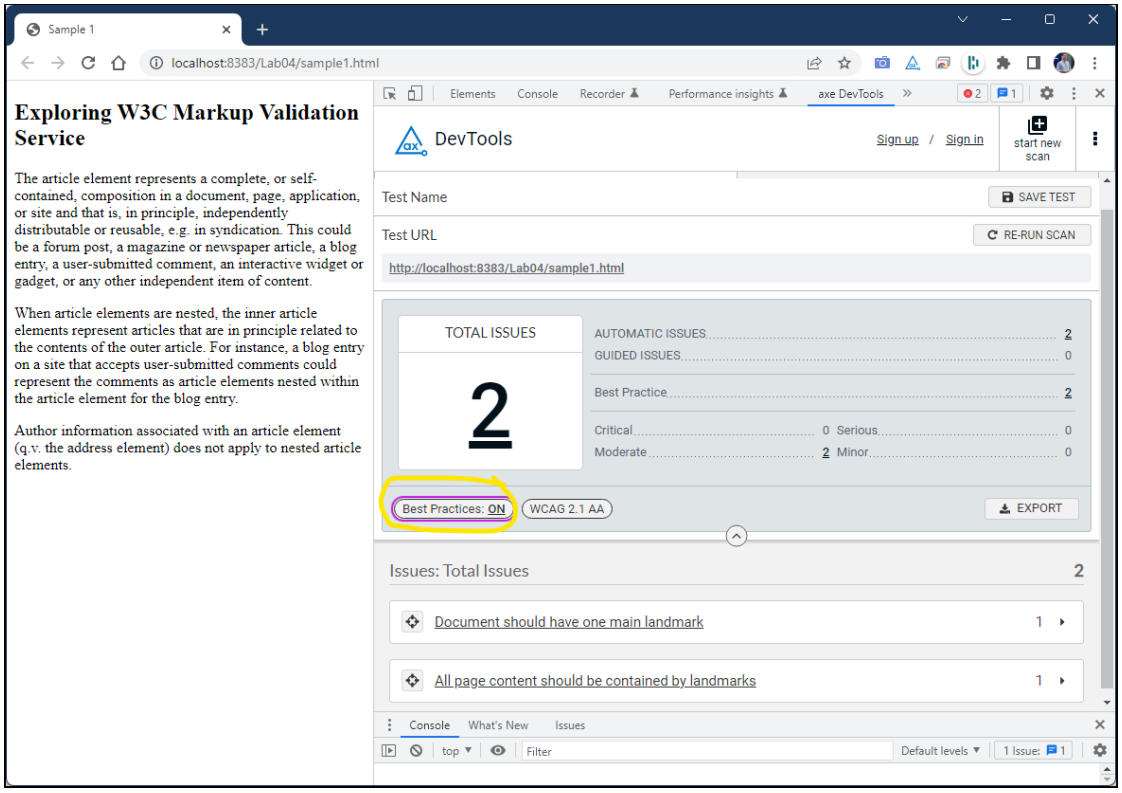
6.2 Now we'll perform accessibility testing on each of the sample files that we fixed in exercise 1. Follow these steps, starting with 'sample1.html':

a. Use the Five Server or similar VS Code extension to view 'sample1.html' in the browser, then activate the Developer Tools (F12). Look for the axe tab and click it to launch the axe tool.

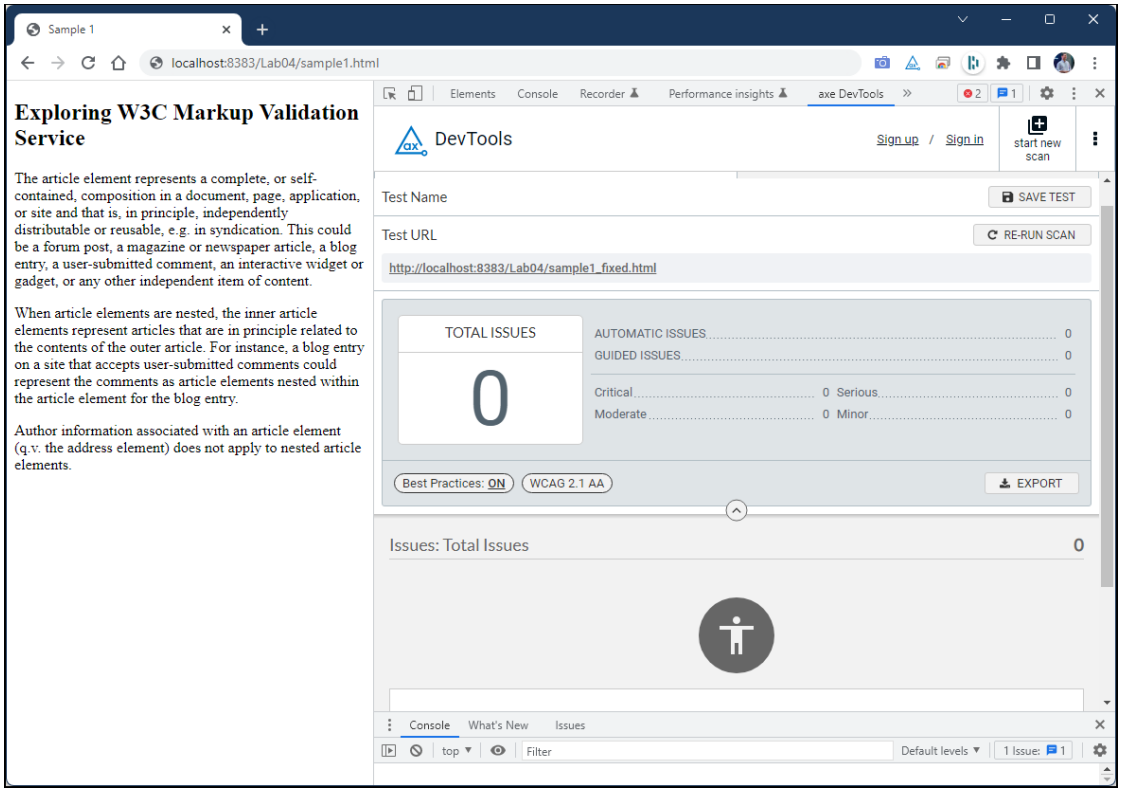
b. Click the 'Scan ALL of my page' button to start the accessibility testing:



c. On the next page, make sure 'Best Practices' is turned ON (it will be OFF by default in the latest version of axe). You should now see two issues, both resulting from the missing <main> element:



d. Edit the file to add the <main> tag, then run the accessibility test again to ensure all issues are resolved:



6.3 Repeat the above steps with the remaining files, 'sample2.html' and 'sample3.html', and fix all issues found.

Tip: if you get stuck, you can click the 'Learn more' link that's provided by axe for each issue.

6.4 Once you've fixed all validation and accessibility issues, submit the three files for your practical assignment.

7. SUBMISSION OF LAB ASSIGNMENT

7.1 In order to receive credit for this Lab assignment, you must submit your completed work to xSiTe LMS before the end of the Lab session. To submit your work:

a. Save all files and close VS Code.

b. In File Explorer, navigate to the location where you saved your project and right-click on the folder name, then select 'Send to -> Compressed (zipped) folder' and ZIP up your entire project. **Note: only .zip format is acceptable, do not use .rar, .7z, or any other format.**

c. In the INF1005 module on xSiTe, go to **Assessments->DropBox** and locate the Dropbox folder corresponding to this Lab. Click the link to open the Dropbox then hit the **Add a**

File button to submit your .zip file. You may also add comments if desired. Be sure to hit **Submit** to complete your submission.

8. STUDY REFERENCES

8.1 **Web Standards**: https://www.w3.org/standards/.

8.2 **Introduction to Web Accessibility**: https://www.w3.org/WAI/fundamentals/accessibility-intro/.

8.3 **Web Content Accessibility Guidelines** (WCAG): https://www.w3.org/WAI/standards-guidelines/wcag/. In particular, you should study the four principles of WCAG, represented by the acronym POUR.